

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Сложение матриц

Матрица представлена как список списков `type Matrix a = [[a]]`. Реализуйте функцию сложения двух матриц одинаковой размерности.

$$C_{ij} = A_{ij} + B_{ij}$$

Обработайте ситуацию, когда размерности матриц не совпадают (верните пустой список или ошибку).

```
1 addMatrices :: Num a    <div data-bbox="77 427 514 447" data-label="Section-Header">

## Задача 2: Умножение на скаляр


```

Реализуйте функцию, которая умножает каждый элемент матрицы на заданное число (скаляр)  $k$ .

$$B_{ij} = k \cdot A_{ij}$$

Используйте функции высшего порядка (`map`).

## Вариант 1

### 题目 1: 矩阵相加.

矩阵用列表的列表表示, 即 `type Matrix a = [[a]]`. 请实现同维度矩阵的加法, 使其满足  $C_{ij} = A_{ij} + B_{ij}$ . 若两矩阵维度不一致, 则返回空列表或抛出错误.

### 解答

基本思路: 先验证两矩阵的维度完全一致, 再用两层 `zipWith` 实现逐行、逐元素相加.

1. Матрица в виде `[[a]]` – это список строк; сложение определяется покомпонентно:  $C_{ij} = A_{ij} + B_{ij}$ .  
矩阵用 `[[a]]` 表示时可看作“行的列表”, 加法按元素定义为  $C_{ij} = A_{ij} + B_{ij}$ .
2. Чтобы выражение `zipWith (zipWith (+))` было корректным для всех позиций, матрицы должны иметь одинаковое число строк и одинаковую длину каждой пары строк.  
要让 `zipWith (zipWith (+))` 在每个位置都合法, 必须满足行数相同且每一对对应行长度相同.
3. Проверка `length a == length b` гарантирует существование попарного соответствия строк  $A_{i*} \leftrightarrow B_{i*}$ .  
检查 `length a == length b` 保证存在逐行对应关系  $A_{i*} \leftrightarrow B_{i*}$ .
4. Далее `zipWith` по строкам сравнивает длины строк, а `and` требует, чтобы все сравнения были истинны.  
随后用 `zipWith` 对每对行比较长度, 再用 `and` 要求所有比较结果都为真.
5. Если размерности совпадают, внешний `zipWith` складывает строки, а внутренний `zipWith (+)` складывает элементы внутри строк.  
当维度一致时, 外层 `zipWith` 逐行处理, 内层 `zipWith (+)` 在行内逐元素相加.
6. Если размерности не совпадают, по условию допускается вернуть пустой список; здесь возвращается `[]`. Сложность равна  $O(nm)$ .  
若维度不一致, 题目允许返回空列表, 这里返回 `[]`. 时间复杂度为  $O(nm)$ .

```
1 type Matrix a = [[a]]
2
3 sameShape :: Matrix a -> Matrix b -> Bool
4 sameShape a b =
5     length a == length b
6     && and (zipWith (\ra rb -> length ra == length rb) a b)
7
8 addMatrices :: Num a => Matrix a -> Matrix a -> Matrix a
9 addMatrices a b
10     | sameShape a b = zipWith (zipWith (+)) a b
11     | otherwise     = []
```

## 题目 2：矩阵乘以标量.

给定标量  $k$ ，请实现把矩阵每个元素都乘以  $k$  的函数，即  $B_{ij} = k \cdot A_{ij}$ . 要求使用高阶函数 `map`.

### 解答

基本思路：用两层 `map` 分别遍历“行”与“行内元素”，把每个元素  $x$  变成  $k \cdot x$ .

1. По определению требуется получить  $B$ , где  $B_{ij} = k \cdot A_{ij}$  для всех  $i, j$ .

按定义要构造矩阵  $B$ ，使得对任意  $i, j$  都有  $B_{ij} = k \cdot A_{ij}$ .

2. Так как матрица – список строк, удобно применить `map` к списку строк и ещё один `map` к элементам каждой строки.

因为矩阵是“行的列表”，所以可以先对行用一次 `map`，再对行内元素用一次 `map`.

3. Внешний `map` перебирает строки  $A_{i*}$  и преобразует каждую строку независимо.

外层 `map` 遍历每一行  $A_{i*}$ ，并独立地变换该行.

4. Внутренний `map (k *)` заменяет каждый элемент  $x$  на  $k \cdot x$ , что точно соответствует формуле.

内层 `map (k *)` 把每个元素  $x$  变为  $k \cdot x$ ，与公式完全一致.

5. Итоговая запись `map (map (k *))` является прямой реализацией определения. Сложность равна  $O(nm)$ .

最终形式 `map (map (k *))` 直接对应定义. 时间复杂度为  $O(nm)$ .

```
1 scaleMatrix :: Num a => a -> Matrix a -> Matrix a
2 scaleMatrix k = map (map (k *))
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Транспонирование

Реализуйте функцию транспонирования матрицы (строки становятся столбцами).

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}^T \longrightarrow \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Постарайтесь реализовать это с помощью функции `zipWith` или `foldr`.

```
1 transpose :: [[a]] -> [[a]]
```

## Задача 2: След матрицы

Следом квадратной матрицы называется сумма элементов главной диагонали.

$$\text{tr}(A) = \sum_{i=1}^n a_{ii}$$

Напишите функцию вычисления следа.

## Вариант 2

### 题目 1: 矩阵转置.

请实现矩阵转置函数, 使行变列、列变行. 尽量用 `zipWith` 或 `foldr` 实现.

### 解答

基本思路: 把转置理解为“按列收集元素”, 用 `foldr` 逐行把元素“拉链式”地插入到各列中, 再截去多余空列.

1. Транспонирование означает, что столбцы становятся строками:  $(A^T)_{ij} = A_{ji}$ .

转置的含义是列变行:  $(A^T)_{ij} = A_{ji}$ .

2. Представим результат как список столбцов, каждый столбец – это список элементов; тогда нужно собрать элементы одной и той же позиции из всех строк.

把结果看作“列的列表”, 每一列是一个列表; 目标是把各行中同一列位置的元素收集到一起.

3. Функция `foldr (zipWith (:)) (repeat [])` обрабатывает строки справа налево и добавляет элементы строки в начало соответствующих столбцов.

`foldr (zipWith (:)) (repeat [])` 从后向前处理行, 并把一行中的元素分别用 `(:)` 加到对应列的表头.

4. Начальное значение `repeat []` даёт бесконечный список пустых столбцов, чтобы `zipWith` всегда имел правую часть нужной длины.

初值 `repeat []` 提供无限个空列, 使得 `zipWith` 总能找到对应的“列容器”来接收元素.

5. После свёртки остаются хвостовые пустые списки; `takeWhile (not . null)` удаляет их и оставляет только реальные столбцы.

折叠后会残留尾部空列; `takeWhile (not . null)` 会截断这些空列, 只保留真实列.

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3   takeWhile (not . null)
4     . foldr (zipWith (:)) (repeat [])
```

## 题目 2：方阵的迹.

方阵迹定义为主对角线元素之和  $\text{tr}(A) = \sum_{i=1}^n a_{ii}$ . 请实现求迹函数.

### 解答

基本思路：迹等于主对角线求和；把第  $i$  行与索引  $i$  配对，取出  $a_{ii}$  再求和.

1. След квадратной матрицы определяется как сумма диагональных элементов:  $\text{tr}(A) = \sum_{i=1}^n a_{ii}$ .

方阵的迹定义为主对角线元素之和:  $\text{tr}(A) = \sum_{i=1}^n a_{ii}$ .

2. В нулевой индексации нужно взять элементы  $a_{00}, a_{11}, \dots$ ; это удобно сделать, сопоставив каждой строке её индекс.

用从 0 开始的下标时，需要取  $a_{00}, a_{11}, \dots$ ，可以把每行与其索引配对实现.

3. `zipWith (\row i -> row !! i) m [0..]` извлекает по одному диагональному элементу из каждой строки.

`zipWith (\row i -> row !! i) m [0..]` 会从第  $i$  行取出第  $i$  个元素.

4. Затем `sum` суммирует полученные элементы и даёт значение следа.

随后用 `sum` 对这些对角元素求和得到迹.

```
1 traceMat :: Num a => [[a]] -> a
2 traceMat m = sum (zipWith (\row i -> row !! i) m [0 ..])
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Умножение матриц

Реализуйте классическое матричное умножение  $A \times B$ .

$$C_{ij} = \sum_k A_{ik} \cdot B_{kj}$$

Помните: число столбцов  $A$  должно совпадать с числом строк  $B$ .

```
1 multMatrices :: Num a    <div> [[a]] -> [[a]] -> [[a]]</div>
```

## Задача 2: Генерация единичной матрицы

Напишите функцию `identity`, которая генерирует единичную матрицу размера  $n \times n$ .

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

## Вариант 3

### 题目 1: 矩阵乘法.

请实现经典矩阵乘法  $C = A \cdot B$ , 满足  $C_{ij} = \sum_k A_{ik} \cdot B_{kj}$ . 要求注意维度兼容:  $A$  的列数必须等于  $B$  的行数.

### 解答

基本思路: 先把  $B$  转置得到所有列, 再对每个  $A$  的行与  $B$  的列做点积, 点积用 `sum (zipWith (*) ...)`.

1. По определению  $C_{ij} = \sum_k A_{ik} B_{kj}$ , то есть элемент результата – скалярное произведение строки и столбца.

按定义  $C_{ij} = \sum_k A_{ik} B_{kj}$ , 也就是结果元素等于 “行与列的点积” .

2. Чтобы получать столбцы  $B_{*j}$  без индексации, удобно взять  $B^T$ : его строки – это столбцы исходной матрицы.

为避免显式索引, 可以先取  $B^T$ , 因为  $B^T$  的行就是  $B$  的列.

3. Для каждой строки `row` из  $A$  и каждой строки `col` из  $B^T$  вычисляем `sum (zipWith (*) row col)`.  
对  $A$  的每行 `row` 和  $B^T$  的每行 `col` 计算 `sum (zipWith (*) row col)`.

4. Размерности совместимы тогда и только тогда, когда число столбцов  $A$  равно числу строк  $B$ ; иначе возвращаем `[]`.

只有当  $A$  的列数等于  $B$  的行数时才能相乘, 否则返回 `[]`.

5. Время работы  $O(nmp)$  для  $n \times m$  и  $m \times p$  матриц.

对  $n \times m$  与  $m \times p$  的矩阵, 时间复杂度为  $O(nmp)$ .

```
1  type Matrix a = [[a]]
2
3  nRows :: Matrix a -> Int
4  nRows = length
5
6  nCols :: Matrix a -> Int
7  nCols []      = 0
8  nCols (r:_) = length r
9
10 transpose :: [[a]] -> [[a]]
11 transpose =
12     takeWhile (not . null)
13     . foldr (zipWith (:)) (repeat [])
14
15 multMatrices :: Num a => Matrix a -> Matrix a -> Matrix a
16 multMatrices a b
17   | null a || null b = []
18   | nCols a /= nRows b = []
19   | otherwise =
20       let bt = transpose b
21       in [ [ sum (zipWith (*) row col) | col <- bt ] | row <- a ]
```



## 题目 2：生成单位矩阵。

请实现 `identity`，生成大小为  $n \times n$  的单位矩阵。

### 解答

基本思路：用两层列表推导枚举  $i, j$ ，当  $i = j$  放 1，否则放 0。

1. Единичная матрица  $I$  задаётся условиями:  $I_{ij} = 1$  при  $i = j$  и  $I_{ij} = 0$  иначе.

单位矩阵满足：当  $i = j$  时  $I_{ij} = 1$ ，否则  $I_{ij} = 0$ 。

2. В Haskell удобно явно перебрать индексы строк и столбцов с помощью вложенного list comprehension.

在 Haskell 中可以用嵌套列表推导显式枚举行、列索引。

3. Условие `if i == j then 1 else 0` непосредственно реализует определение элемента.

用 `if i == j then 1 else 0` 直接实现元素定义。

4. Получаем квадратную матрицу размера  $n \times n$ . Время работы  $O(n^2)$ .

最终得到大小为  $n \times n$  的方阵，时间复杂度为  $O(n^2)$ 。

```
1 identity :: Num a => Int -> Matrix a
2 identity n =
3   [ [ if i == j then 1 else 0 | j <- [0 .. n - 1] ]
4     | i <- [0 .. n - 1]
5   ]
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

Задача 1: Матрица на вектор

Реализуйте умножение матрицы на вектор-столбец. Вектор представлен как список [a].

$$\vec{y} = A\vec{x}$$

1

`multMatVec :: Num a => [[a]] -> [a] -> [a]`

Задача 2: Проверка на симметричность

Напишите предикат, который проверяет, является ли квадратная матрица симметричной, то есть  $A = A^T$ .

$$A_{ij} = A_{ji}$$

## Вариант 4

### 题目 1: 矩阵乘向量.

请实现矩阵与列向量的乘法  $\vec{y} = A\vec{x}$ . 向量用 `[a]` 表示.

### 解答

基本思路: 把  $y = Ax$  看作 “每行与向量的点积”, 对每一行用 `zipWith (*)` 与 `sum`, 再用 `map` 处理所有行.

1. Умножение матрицы на вектор даёт  $y_i = \sum_j A_{ij}x_j$ , то есть скалярное произведение  $i$ -й строки и вектора.

矩阵乘向量满足  $y_i = \sum_j A_{ij}x_j$ , 即第  $i$  行与向量的点积.

2. Для фиксированной строки `row` можно вычислить  $\sum_j \text{row}_j x_j$  как `sum (zipWith (*) row x)`.

固定一行 `row` 时, 可用 `sum (zipWith (*) row x)` 计算  $\sum_j \text{row}_j x_j$ .

3. Внешний `map` применяет это вычисление ко всем строкам матрицы и формирует список  $[y_0, y_1, \dots]$ .

外层 `map` 把该计算作用到每一行, 得到结果向量  $[y_0, y_1, \dots]$ .

4. Если длины строки и вектора различаются, `zipWith` усечёт по меньшей длине; при необходимости можно дополнительно проверить размерность.

若行长度与向量长度不等, `zipWith` 会按较短者截断; 如需严格检查可额外验证维度.

```
1 multMatVec :: Num a => [[a]] -> [a] -> [a]
2 multMatVec a x = map (\row -> sum (zipWith (*) row x)) a
```

## 题目 2：判断矩阵是否对称.

请写出谓词判断一个方阵是否对称，即  $A = A^T$ ，等价于  $A_{ij} = A_{ji}$ .

### 解答

基本思路：先验证是方阵，再比较  $A$  与  $A^T$  是否相等；对称条件等价于  $A_{ij} = A_{ji}$ .

1. Симметричность означает  $A = A^T$ , а это возможно только для квадратных матриц.  
矩阵对称要求  $A = A^T$ ，因此必须先保证它是方阵.
2. Проверка `isRectangular` гарантирует, что все строки одной длины, то есть матрица имеет корректную прямоугольную форму.  
`isRectangular` 用来保证每行等长，即矩阵是合法的矩形结构.
3. Проверка `length m == length (head m)` даёт равенство числа строк и столбцов, то есть квадратность.  
检查 `length m == length (head m)` 使行数等于列数，从而保证方阵.
4. После этого достаточно сравнить  $m$  и `transpose m`: равенство списков эквивалентно  $A_{ij} = A_{ji}$  для всех позиций.  
随后比较  $m$  与 `transpose m` 是否相等；列表相等就等价于所有位置满足  $A_{ij} = A_{ji}$ .

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3     takeWhile (not . null)
4         . foldr (zipWith (:)) (repeat [])
5
6 isRectangular :: [[a]] -> Bool
7 isRectangular [] = True
8 isRectangular (r:rs) = all (\row -> length row == length r) rs
9
10 isSquare :: [[a]] -> Bool
11 isSquare m = isRectangular m && length m == (if null m then 0 else length (head m))
12
13 isSymmetric :: Eq a => [[a]] -> Bool
14 isSymmetric m = isSquare m && m == transpose m
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Минор элемента

Напишите функцию `removeRowCol`, которая удаляет  $i$ -ю строку и  $j$ -й столбец из матрицы. Это используется для вычисления миноров.

```
1 removeRowCol :: Int → Int → [[a]] → [[a]]
```

## Задача 2: Определитель (Рекурсивно)

Реализуйте вычисление определителя квадратной матрицы, используя разложение Лапласа по первой строке.

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} M_{1j}$$

## Вариант 5

### 题目 1: 删除指定行与列.

请实现 `removeRowCol`, 从矩阵中删除第  $i$  行与第  $j$  列, 用于计算余子式.

### 解答

基本思路: 用带索引的过滤实现删除第  $i$  行与第  $j$  列: 先过滤行, 再在每行中过滤列.

1. Минор  $M_{ij}$  получается удалением  $i$ -й строки и  $j$ -го столбца из матрицы.

余子式对应的子矩阵可由删除第  $i$  行与第  $j$  列得到.

2. С помощью `zip m [0..]` можно получить пары  $(row, i)$  и отфильтровать строку с индексом  $i$ .

用 `zip m [0..]` 给每行附加索引, 再过滤掉索引为  $i$  的那一行.

3. Для каждой оставшейся строки используем `zip row [0..]` и фильтруем элементы с индексом  $j$ .

对保留的每一行再用 `zip row [0..]` 给列加索引, 过滤掉索引为  $j$  的元素.

4. Результат – новая матрица без выбранной строки и столбца, пригодная для вычисления определителя по разложению.

最终得到删除指定行列后的新矩阵, 可用于拉普拉斯展开的递归计算.

```
1 removeRowCol :: Int -> Int -> [[a]] -> [[a]]
2 removeRowCol i j m =
3   [ [ x | (x, cj) <- zip row [0 ..], cj /= j ]
4     | (row, ri) <- zip m [0 ..], ri /= i
5   ]
```

## 题目 2：递归计算行列式。

请用第一行拉普拉斯展开实现行列式： $\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} M_{1j}$ .

### 解答

基本思路：按第一行做拉普拉斯展开：对每列  $j$  取  $a_{1j}$ ，乘符号  $(-1)^{1+j}$  与余子式行列式并求和。

1. Разложение Лапласа по первой строке:  $\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(M_{1j})$ .

第一行拉普拉斯展开： $\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(M_{1j})$ .

2. В нулевой индексации столбец  $j$  соответствует формуле со знаком  $(-1)^j$ , поэтому удобно взять `cof j = if even j then 1 else (-1)`.

使用从 0 开始的列索引时，符号因子等价于  $(-1)^j$ ，可用 `even` 判断正负。

3. Для каждого  $j$  строим минор `removeRowCol 0 j m` и рекурсивно вычисляем его определитель.

对每个  $j$  构造 `removeRowCol 0 j m`，并递归计算其行列式。

4. База рекурсии:  $0 \times 0$  матрица имеет  $\det = 1$ , а  $1 \times 1 - \det = [[x]] = x$ .

递归基：空矩阵定义  $\det = 1$ ，而  $1 \times 1$  矩阵满足  $\det([[x]]) = x$ 。

5. Метод очень дорогой: сложность порядка  $O(n!)$ , поэтому применим только для малых  $n$ .

该方法复杂度约为  $O(n!)$ ，只适合很小的  $n$ 。

```
1 det :: Num a => [[a]] -> a
2 det []      = 1
3 det [[x]]   = x
4 det m       =
5   let firstRow = head m
6       n        = length firstRow
7       cof j    = if even j then 1 else (-1)
8   in sum
9     [ cof j * (firstRow !! j) * det (removeRowCol 0 j m)
10     | j <- [0 .. n - 1]
11     ]
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Диагональная матрица

Реализуйте функцию `fromDiag`, которая принимает список чисел и строит квадратную диагональную матрицу, где эти числа стоят на главной диагонали, а остальные – нули.

## Задача 2: Тип данных `Matrix`

Вместо списков списков создайте свой тип данных `Matrix` `a`, который хранит размерность и данные (например, в плоском списке).

```
1 data Matrix a = Matrix { rows :: Int, cols :: Int, mdata :: [a] }
```

Реализуйте для него инстанс класса `Show` для красивого вывода в виде таблицы.



## Вариант 6

### 题目 1: 由对角元素构造对角矩阵.

给定一个列表  $[d_0, d_1, \dots, d_{n-1}]$ , 构造一个  $n \times n$  的对角矩阵, 使主对角线为这些数, 其余元素为 0.

### 解答

基本思路: 设  $n = \text{length } ds$ , 用两层列表推导生成  $n \times n$  矩阵, 满足  $i = j$  时取  $ds!!i$ , 否则取 0.

1. Диагональная матрица определяется так:  $A_{ii} = d_i$ , а  $A_{ij} = 0$  при  $i \neq j$ .

对角矩阵满足:  $A_{ii} = d_i$ , 且当  $i \neq j$  时  $A_{ij} = 0$ .

2. Размер  $n$  равен длине списка диагональных элементов:  $n = \text{length } ds$ .

矩阵大小由对角元素列表长度确定:  $n = \text{length } ds$ .

3. Вложенный list comprehension перебирает индексы  $i, j$  и заполняет элемент по условию `if i == j` ....

嵌套列表推导遍历  $i, j$ , 用 `if i == j` ... 按条件填充元素.

4. Так получаем корректную диагональную матрицу. Сложность  $O(n^2)$ .

因此得到正确的对角矩阵, 时间复杂度为  $O(n^2)$ .

```
1 fromDiag :: Num a => [a] -> [[a]]
2 fromDiag ds =
3   let n = length ds
4   in [ [ if i == j then ds !! i else 0 | j <- [0 .. n - 1] ]
5       | i <- [0 .. n - 1]
6     ]
```

## 题目 2：自定义 Matrix 类型并实现 Show.

定义 `Matrix a = Matrix { rows :: Int, cols :: Int, mdata :: [a] }`, 其中 `mdata` 为按行展开的扁平列表. 请为其实现 `Show`, 使其以表格样式输出.

### 解答

基本思路: 用 `rows`、`cols` 记录维度, `mdata` 以按行展开存储; 索引公式为  $i \cdot \text{cols} + j$ , 打印时按行切回二维输出.

1. Плоское хранение означает, что элемент  $(i, j)$  лежит по индексу  $i \cdot c + j$ , где  $c = \text{cols}$ .

按行展开存储时, 元素  $(i, j)$  的线性下标为  $i \cdot c + j$ , 其中  $c = \text{cols}$ .

2. Функция `index` реализует доступ по этой формуле и извлекает элемент из списка `mdata`.

`index` 按该公式从 `mdata` 中取出对应元素.

3. В `Show`-инстансе мы строим строки: для каждого  $i$  печатаем элементы  $j = 0 \dots c - 1$  и соединяем их через `unwords`.

在 `Show` 实例中, 对每个  $i$  构造一行, 遍历  $j = 0 \dots c - 1$  并用 `unwords` 连接.

4. Затем `unlines` объединяет строки в многострочный текст, что даёт табличный вывод.

再用 `unlines` 把各行连接为多行文本, 从而得到表格式输出.

5. Размерности `rows` и `cols` определяют формат вывода; при некорректной длине `mdata` возможна ошибка индексации.

`rows` 与 `cols` 决定输出形状; 若 `mdata` 长度不正确则可能触发索引错误.

```
1 data Matrix a = Matrix
2   { rows  :: Int
3   , cols  :: Int
4   , mdata :: [a]
5   }
6
7 index :: Matrix a -> Int -> Int -> a
8 index (Matrix _ c xs) i j = xs !! (i * c + j)
9
10 instance Show a => Show (Matrix a) where
11   show m@(Matrix r c _) =
12     unlines
13       [ unwords [ show (index m i j) | j <- [0 .. c - 1] ]
14       | i <- [0 .. r - 1]
15     ]
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

**Задача 1: Скалярное произведение векторов**

Векторы – это строки матрицы. Реализуйте функцию скалярного произведения (dot product) двух списков.

$$\vec{a} \cdot \vec{b} = a_1 b_1 + \dots + a_n b_n$$

**Задача 2: Возведение матрицы в степень**

Реализуйте функцию возведения квадратной матрицы  $A$  в целую неотрицательную степень  $k$ .

$$A^0 = I, \quad A^k = \underbrace{A \times \dots \times A}_k$$

Используйте алгоритм быстрого возведения в степень (как для чисел).

## Вариант 7

### 题目 1: 向量点积.

请实现两个列表的点积  $\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$ .

### 解答

基本思路：点积按定义是逐项相乘再求和，用 `zipWith (*)` 配对相乘，再用 `sum` 汇总。

1. Скалярное произведение:  $\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$ .

向量点积为  $\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$ .

2. `zipWith (*) xs ys` строит список произведений  $[a_1 b_1, \dots, a_n b_n]$  (по меньшей длине).

`zipWith (*) xs ys` 生成逐项乘积列表  $[a_1 b_1, \dots, a_n b_n]$  (按较短长度截断)。

3. `sum` суммирует эти произведения и возвращает итоговое значение.

`sum` 对乘积列表求和得到点积结果。

4. Сложность  $O(n)$ .

时间复杂度为  $O(n)$ .

```
1 dot :: Num a => [a] -> [a] -> a
2 dot xs ys = sum (zipWith (*) xs ys)
```

## 题目 2：矩阵快速幂.

请实现方阵  $A$  的非负整数幂  $A^k$ ，满足  $A^0 = I$ . 要求使用快速幂.

### 解答

基本思路：利用快速幂思想： $k = 0$  返回单位矩阵； $k$  偶数时递归计算  $(A^2)^{k/2}$ ，奇数时计算  $A \cdot A^{k-1}$ .

1. Определение:  $A^0 = I$ , а для  $k > 0$  матрица умножается сама на себя  $k$  раз.

定义为  $A^0 = I$ ，当  $k > 0$  时为  $A$  的  $k$  次连乘.

2. Быстрое возведение: если  $k$  чётно, то  $A^k = (A \cdot A)^{k/2}$ ; если  $k$  нечётно, то  $A^k = A \cdot A^{k-1}$ .

快速幂：若  $k$  为偶数，则  $A^k = (A \cdot A)^{k/2}$ ；若为奇数，则  $A^k = A \cdot A^{k-1}$ .

3. База: при  $k = 0$  возвращаем  $I$  того же размера, что и  $A$ .

递归基：当  $k = 0$  时返回与  $A$  同阶的单位矩阵  $I$ .

4. Каждый шаг либо уменьшает  $k$  вдвое, либо уменьшает на 1, поэтому число умножений  $O(\log k)$ .

每一步要么把  $k$  减半，要么减一，因此矩阵乘法次数为  $O(\log k)$ .

5. Корректность умножения зависит от корректной реализации `multMatrices` и `identity`.

正确性依赖于 `multMatrices` 与 `identity` 的正确实现.

```
1 type Matrix a = [[a]]
2
3 transpose :: [[a]] -> [[a]]
4 transpose =
5     takeWhile (not . null)
6         . foldr (zipWith (:)) (repeat [])
7
8 multMatrices :: Num a => Matrix a -> Matrix a -> Matrix a
9 multMatrices a b =
10     let bt = transpose b
11     in [ [ sum (zipWith (*) row col) | col <- bt ] | row <- a ]
12
13 identity :: Num a => Int -> Matrix a
14 identity n =
15     [ [ if i == j then 1 else 0 | j <- [0 .. n - 1] ]
16     | i <- [0 .. n - 1]
17     ]
18
19 powMatrix :: (Num a, Integral k) => Matrix a -> k -> Matrix a
20 powMatrix a k
21     | k < 0     = error "powMatrix: negative exponent"
22     | k == 0    = identity (length a)
23     | even k    = powMatrix (multMatrices a a) (k `div` 2)
24     | otherwise = multMatrices a (powMatrix a (k - 1))
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Разреженные матрицы

Для экономии памяти разреженные матрицы часто хранят как список координат ненулевых элементов: `type SparseMatrix a = [((Int, Int), a)]`. Реализуйте функцию сложения двух таких матриц.

```
1 addSparse :: (Eq a, Num a) => [((Int, Int), a)] -> [((Int, Int), a)] -> [((Int, Int), a)]
```

## Задача 2: Плотная в разреженную

Напишите функцию преобразования обычной матрицы `[[a]]` в формат `SparseMatrix a`, отбрасывая нули.

## Вариант 8

### 题目 1: 稀疏矩阵相加.

稀疏矩阵用非零元素坐标列表表示: `type SparseMatrix a = [((Int, Int), a)]`. 请实现两稀疏矩阵相加, 并把相同坐标处的值相加; 若结果为 0 则应丢弃该项.

### 解答

基本思路: 把稀疏矩阵看作“坐标到数值”的映射, 先用 `fromListWith (+)` 合并重复坐标, 再用 `unionWith (+)` 做两矩阵相加, 最后过滤掉值为 0 的项.

1. Формат `SparseMatrix a = [((Int,Int),a)]` хранит только ненулевые элементы с их координатами.

稀疏矩阵 `[((Int,Int),a)]` 仅存储非零元素及其坐标.

2. Сначала удобно агрегировать список в Map по ключу-координате, чтобы одинаковые координаты суммировались через `fromListWith (+)`.

先用 Map 以坐标为键聚合列表, 通过 `fromListWith (+)` 把重复坐标的值相加.

3. Затем `unionWith (+)` складывает два отображения, реализуя покоординатное сложение.

随后用 `unionWith (+)` 合并两张映射表, 实现逐坐标相加.

4. После сложения элементы со значением 0 следует отбросить, поэтому фильтруем  $v \neq 0$ .

相加后若某坐标的结果为 0, 应丢弃该项, 因此过滤  $v \neq 0$ .

5. Возвращаем результат обратно в список через `toList`.

最后用 `toList` 把映射转回列表形式.

```
1 import qualified Data.Map.Strict as M
2
3 type SparseMatrix a = [((Int, Int), a)]
4
5 addSparse :: (Eq a, Num a) => SparseMatrix a -> SparseMatrix a -> SparseMatrix a
6 addSparse xs ys =
7   [ (p, v) | (p, v) <- M.toList m, v /= 0 ]
8   where
9     mx = M.fromListWith (+) xs
10    my = M.fromListWith (+) ys
11    m  = M.unionWith (+) mx my
```

## 题目 2: 稠密矩阵转稀疏矩阵.

将普通矩阵 `[[a]]` 转为 `SparseMatrix a`, 要求丢弃所有值为 0 的元素.

### 解答

基本思路: 用两层 `zip` 枚举行索引与列索引, 对每个元素  $x$  若  $x \neq 0$  就输出其坐标与数值.

1. В плотной матрице `[[a]]` координаты можно получить, пронумеровав строки и элементы в строке:  
`zip m [0..]` и `zip row [0..]`.

把稠密矩阵 `[[a]]` 转稀疏时, 可用 `zip m [0..]` 枚举行索引, 再用 `zip row [0..]` 枚举列索引.

2. Для каждого элемента  $x$  оставляем пару  $((i, j), x)$  только если  $x \neq 0$ .

对每个元素  $x$ , 仅当  $x \neq 0$  时保留  $((i, j), x)$ .

3. Так получаем список всех ненулевых элементов в формате `SparseMatrix`.

从而得到所有非零元素的 `SparseMatrix` 表示.

```
1 denseToSparse :: (Eq a, Num a) => [[a]] -> SparseMatrix a
2 denseToSparse m =
3   [ ((i, j), x)
4     | (row, i) <- zip m [0 ..]
5       , (x, j)   <- zip row [0 ..]
6       , x /= 0
7     ]
```



Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

### Задача 1: Правило Крамера

Дана система линейных уравнений  $A\vec{x} = \vec{b}$ . Реализуйте функцию решения системы методом Крамера.

$$x_i = \frac{\det(A_i)}{\det(A)}$$

где  $A_i$  – матрица  $A$  с замененным  $i$ -м столбцом на вектор  $\vec{b}$ .

### Задача 2: Норма вектора

Напишите функцию для вычисления Евклидовой нормы вектора (строки матрицы).

$$\|\vec{v}\| = \sqrt{v_1^2 + \dots + v_n^2}$$

## Вариант 9

### 题目 1: 克拉默法则解线性方程组.

给定线性方程组  $A\vec{x} = \vec{b}$ . 请用克拉默法则实现求解:  $x_i = \frac{\det(A_i)}{\det(A)}$ , 其中  $A_i$  是把  $A$  的第  $i$  列替换为  $\vec{b}$  得到的矩阵.

### 解答

基本思路: 按克拉默法则: 先算  $\det(A)$ , 若为 0 则返回 `Nothing`; 否则对每列构造  $A_i$  并计算  $x_i = \frac{\det(A_i)}{\det(A)}$ .

1. Правило Крамера применимо, когда  $\det(A) \neq 0$ ; тогда решение единственно и  $x_i = \frac{\det(A_i)}{\det(A)}$ .

克拉默法则要求  $\det(A) \neq 0$ ; 此时解唯一, 且  $x_i = \frac{\det(A_i)}{\det(A)}$ .

2. Сначала вычисляем  $\det(A)$ ; если он равен нулю, возвращаем `Nothing`.

先计算  $\det(A)$ ; 若等于 0 则返回 `Nothing`.

3. Матрица  $A_i$  получается заменой  $i$ -го столбца на вектор  $b$ ; это делается функцией `replaceCol`.

$A_i$  通过把第  $i$  列替换为向量  $b$  得到, 对应实现为 `replaceCol`.

4. Затем строим список  $\left[ \frac{\det(A_0)}{\det(A)}, \dots \right]$  с помощью list comprehension.

再用列表推导计算  $\left[ \frac{\det(A_0)}{\det(A)}, \dots \right]$  得到解向量.

5. Так как определитель вычисляется рекурсивно, метод имеет высокую сложность и подходит для малых размерностей.

由于行列式采用递归展开计算, 复杂度很高, 仅适合小规模方程组.

```

1  removeRowCol :: Int -> Int -> [[a]] -> [[a]]
2  removeRowCol i j m =
3      [ [ x | (x, cj) <- zip row [0 ..], cj /= j ]
4        | (row, ri) <- zip m [0 ..], ri /= i
5      ]
6
7  det :: Num a => [[a]] -> a
8  det []      = 1
9  det [[x]]   = x
10 det m       =
11     let firstRow = head m
12         n        = length firstRow
13         cof j     = if even j then 1 else (-1)
14     in sum
15         [ cof j * (firstRow !! j) * det (removeRowCol 0 j m)
16         | j <- [0 .. n - 1]
17       ]
18
19 replaceCol :: Int -> [a] -> [[a]] -> [[a]]
20 replaceCol j col m =
21     zipWith (\row x -> take j row ++ [x] ++ drop (j + 1) row) m col
22
23 isRectangular :: [[a]] -> Bool
24 isRectangular []      = True
25 isRectangular (r:rs) = all (\row -> length row == length r) rs
26
27 isSquare :: [[a]] -> Bool
28 isSquare m = isRectangular m && length m == (if null m then 0 else length (head m))
29
30 cramerSolve :: (Fractional a, Eq a) => [[a]] -> [a] -> Maybe [a]
31 cramerSolve a b
32   | not (isSquare a)      = Nothing
33   | length b /= n         = Nothing
34   | detA == 0             = Nothing
35   | otherwise             = Just [ det (replaceCol i b a) / detA | i <- [0 .. n - 1] ]
36   where
37     n      = length a
38     detA   = det a

```

## 题目 2：向量的欧几里得范数.

实现欧几里得范数  $\|\vec{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$ .

### 解答

基本思路：欧几里得范数按定义是平方和开方：先平方再求和，最后 `sqrt`.

1. Евклидова норма:  $\|\vec{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$ .

欧几里得范数:  $\|\vec{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$ .

2. `map (\x -> x*x) v` даёт список квадратов элементов.

`map (\x -> x*x) v` 得到每个分量的平方列表.

3. `sum` суммирует квадраты, а `sqrt` берёт квадратный корень от суммы.

用 `sum` 求平方和，再用 `sqrt` 开方即可.

```
1 euclidNorm :: Floating a => [a] -> a
2 euclidNorm v = sqrt (sum (map (\x -> x * x) v))
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Элементарные преобразования

Для реализации метода Гаусса напишите функцию, которая вычитает из строки  $i$  строку  $j$ , умноженную на коэффициент  $c$ .

$$R_i \leftarrow R_i - c \cdot R_j$$

```
1 subRow :: Num a    [a] -> [a] -> a -> [a]
```

## Задача 2: Класс Functor для Матрицы

Оберните список списков в `newtype Mat a = Mat [[a]]`. Сделайте `Mat` экземпляром класса `Functor`. Это позволит применять функцию (например, `+1`) ко всем элементам матрицы сразу через `fmap`.

```
1 newtype Mat a = Mat [[a]] deriving Show
2 -- instance Functor Mat where \dots
```

## Вариант 10

### 题目 1: 高斯消元的行变换.

实现  $R_i \leftarrow R_i - c \cdot R_j$ . 函数参数按 `subRow rowI rowJ c` 理解.

### 解答

基本思路: 把行变换写成逐元素运算: 对每对元素  $(x, y)$  计算  $x - cy$ , 用 `zipWith` 实现.

1. Элементарное преобразование  $R_i \leftarrow R_i - c \cdot R_j$  выполняется поэлементно.

行变换  $R_i \leftarrow R_i - c \cdot R_j$  是逐元素执行的.

2. Если  $R_i = [x_1, \dots]$  и  $R_j = [y_1, \dots]$ , то результат  $[x_1 - cy_1, x_2 - cy_2, \dots]$ .

若  $R_i = [x_1, \dots]$ 、 $R_j = [y_1, \dots]$ , 则结果为  $[x_1 - cy_1, x_2 - cy_2, \dots]$ .

3. `zipWith (\x y -> x - c*y)` реализует именно это соответствие.

`zipWith (\x y -> x - c*y)` 正好对应逐项计算.

4. Сложность  $O(m)$  для строки длины  $m$ .

对长度为  $m$  的行, 时间复杂度为  $O(m)$ .

```
1 subRow :: Num a => [a] -> [a] -> a -> [a]
2 subRow rowI rowJ c = zipWith (\x y -> x - c * y) rowI rowJ
```

## 题目 2: 为 Mat 实现 Functor.

定义 `newtype Mat a = Mat [[a]]` 并实现 `Functor`, 使 `fmap f` 能把  $f$  作用到矩阵每个元素上.

### 解答

基本思路: `Functor` 的核心是把函数作用到结构里的每个元素; 对 `Mat [[a]]` 只需对外层与内层各做一次 `map`.

1. Тип `Mat a` оборачивает `[[a]]`; экземпляр `Functor` должен уметь применять функцию к каждому элементу.

`Mat a` 包装了 `[[a]]`; 作为 `Functor`, 需要能把函数作用到每个元素.

2. Функция `fmap f` должна сохранить форму матрицы и заменить каждый элемент  $x$  на  $f(x)$ .

`fmap f` 应保持矩阵形状不变, 并把每个元素  $x$  替换为  $f(x)$ .

3. Внешний `map` идёт по строкам, внутренний `map` – по элементам строки: `map (map f)`.

外层 `map` 遍历行, 内层 `map` 遍历行内元素: `map (map f)`.

4. Такое определение удовлетворяет законам функторов: `fmap id = id` и `fmap(g ∘ f) = fmap g ∘ fmap f`.

该定义满足函子定律: `fmap id = id`, 以及 `fmap(g ∘ f) = fmap g ∘ fmap f`.

```
1 newtype Mat a = Mat [[a]] deriving Show
2
3 instance Functor Mat where
4     fmap f (Mat xss) = Mat (map (map f) xss)
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Map + Filter над списком

Дан список целых чисел. Постройте новый список, содержащий квадраты только тех чисел, которые нечётные и по модулю больше 10. Используйте `filter` и `map`.

```
1 oddBigSquares :: [Int] → [Int]
```

## Задача 2: Fold для проверки свойства

Реализуйте функцию, которая проверяет, что список строго возрастающий. Запрещено использовать явную рекурсию; используйте `foldl` или `foldr`.

```
1 isStrictlyIncreasing :: Ord a    [a] → Bool
```



## Вариант 11

### 题目 1: filter + map.

给定整数列表，筛选出所有满足“奇数且绝对值大于 10”的元素，并返回它们的平方列表。要求使用 `filter` 与 `map`。

### 解答

基本思路：先用 `filter` 选出“奇数且绝对值大于 10”的元素，再用 `map` 求平方，最后得到新列表。

1. Нужно оставить элементы, удовлетворяющие предикату:  $\text{odd}(x)$  и  $|x| > 10$ .

需要筛选满足“奇数且  $|x| > 10$ ”的元素。

2. `filter (\x -> odd x && abs x > 10)` реализует фильтрацию без явной рекурсии.

`filter (\x -> odd x && abs x > 10)` 在无显式递归下完成筛选。

3. После фильтрации `map (\x -> x*x)` вычисляет квадраты оставшихся чисел.

筛选后用 `map (\x -> x*x)` 计算平方。

4. Композиция `map ... . filter ...` даёт компактное решение.

用 `map ... . filter ...` 进行函数组合即可得到简洁实现。

```
1 oddBigSquares :: [Int] -> [Int]
2 oddBigSquares =
3   map (\x -> x * x) . filter (\x -> odd x && abs x > 10)
```

## 题目 2: 用 fold 检查严格递增.

实现 `isStrictlyIncreasing` 检查列表是否严格递增. 禁止显式递归, 要求使用 `foldl` 或 `foldr`.

### 解答

基本思路: 用 `foldl` 在遍历过程中携带“上一个元素”和“当前是否仍严格递增”的状态, 最终返回状态中的布尔值.

1. Строго возрастающий список удовлетворяет  $x_1 < x_2 < \dots < x_n$ .

严格递增列表满足  $x_1 < x_2 < \dots < x_n$ .

2. При свёртке удобно хранить пару  $(prev, ok)$ , где  $prev$  – предыдущий элемент (`Maybe`), а  $ok$  – накопленная проверка.

折叠时可维护二元状态  $(prev, ok)$ :  $prev$  为上一个元素 (`Maybe`),  $ok$  为累计判断结果.

3. Для первого элемента устанавливаем  $prev = \text{Just } x$  и не меняем  $ok$ .

遇到第一个元素时设置  $prev = \text{Just } x$ ,  $ok$  保持不变.

4. Для каждого следующего  $x$  обновляем  $ok := ok \wedge (p < x)$  и  $prev := \text{Just } x$ .

对后续元素  $x$ , 更新  $ok := ok \wedge (p < x)$ , 并令  $prev := \text{Just } x$ .

5. В конце берём вторую компоненту пары; так мы избегаем явной рекурсии.

最终取状态的第二个分量作为答案, 从而避免显式递归.

```
1 isStrictlyIncreasing :: Ord a => [a] -> Bool
2 isStrictlyIncreasing xs = snd (foldl step (Nothing, True) xs)
3 where
4     step (Nothing, ok) x      = (Just x, ok)
5     step (Just p, ok) x      = (Just x, ok && p < x)
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Разбиение списка (Chunking)

Напишите функцию, разбивающую список на подсписки длины  $k$  (последний может быть короче). Используйте `unfoldr` или рекурсию.

```
1 chunksOf :: Int -> [a] -> [[a]]
```

## Задача 2: Скользящее среднее (Matrix/List)

Для списка чисел вычислите скользящее среднее окна длины  $k$ :

$$m_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i+j}$$

Используйте `map + take/drop` или `zipWith`.

```
1 movingAverage :: Fractional a -> Int -> [a] -> [a]
```

## Вариант 12

### 题目 1: 把列表按固定长度分块.

实现 `chunksOf k xs`, 把列表分成若干个长度为  $k$  的子列表, 最后一块可不足  $k$ . 允许用递归或 `unfoldr`.

### 解答

基本思路: 每次取前  $k$  个元素作为一块, 再递归处理剩余部分; 空列表结束; 当  $k \leq 0$  视为非法输入.

1. Разбиение на чанки означает: первый чанк `take k xs`, остаток – `drop k xs`.

分块的核心是: 第一块取 `take k xs`, 剩余部分为 `drop k xs`.

2. Если список пуст, результат пуст: это база рекурсии.

当列表为空时返回空列表, 这是递归基.

3. Если  $k \leq 0$ , длина чанка не имеет смысла, поэтому корректно сообщить об ошибке.

若  $k \leq 0$ , 块长度无意义, 因此直接报错更合理.

4. Рекурсивное определение точно соответствует математическому разбиению списка на подряд идущие подсписки.

递归定义与把列表按顺序分割成连续子列表的数学描述一致.

```
1 chunksOf :: Int -> [a] -> [[a]]
2 chunksOf k _ | k <= 0 = error "chunksOf: k must be positive"
3 chunksOf _ []       = []
4 chunksOf k xs        = take k xs : chunksOf k (drop k xs)
```

## 题目 2：滑动平均.

对列表  $[x_0, x_1, \dots]$  计算窗口长度为  $k$  的滑动平均  $m_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i+j}$ . 要求使用 `map` 以及 `take/drop` 或 `zipWith`.

### 解答

基本思路：先生成所有长度为  $k$  的连续窗口，再对每个窗口求平均；窗口可由 `tails` 产生后对每个后缀取 `take k`.

1. Скользящее среднее определяется как  $m_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i+j}$ .

滑动平均定义为  $m_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i+j}$ .

2. Функция `tails xs` перечисляет все суффиксы списка, то есть  $[x_i, x_{i+1}, \dots]$  для каждого  $i$ .  
`tails xs` 会列出所有后缀，即每个  $i$  的  $[x_i, x_{i+1}, \dots]$ .
3. Для каждого суффикса берём первые  $k$  элементов: `map (take k) (tails xs)` – это все окна кандидаты.  
对每个后缀取前  $k$  项: `map (take k) (tails xs)` 得到所有候选窗口.
4. Неполные окна (длины  $< k$ ) отбрасываем через `filter (\w -> length w == k)`.  
长度不足  $k$  的窗口用 `filter (\w -> length w == k)` 过滤掉.
5. Далее `map (\w -> sum w / fromIntegral k)` вычисляет среднее каждого окна.  
最后用 `map (\w -> sum w / fromIntegral k)` 计算每个窗口的平均值.

```
1 import Data.List (tails)
2
3 movingAverage :: Fractional a => Int -> [a] -> [a]
4 movingAverage k xs
5   | k <= 0      = []
6   | otherwise   =
7     map (\w -> sum w / fromIntegral k)
8       (filter (\w -> length w == k) (map (take k) (tails xs)))
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Поэлементное отображение с проверкой размерностей

Матрицы представлены как списки списков. Реализуйте функцию, которая применяет бинарную функцию  $f : a \rightarrow b \rightarrow c$  к соответствующим элементам двух матриц одинаковой размерности:

$$C_{ij} = f(A_{ij}, B_{ij})$$

Если размерности матриц не совпадают (разное число строк или различная длина хотя бы одной пары строк), функция должна вернуть `Nothing`.

Требования:

- использовать `zipWith` для строк;
- не использовать явную рекурсию;
- аккуратно обработать ошибку размерностей.

```
1 zipWithMatSafe :: (a -> b -> c) -> [[a]] -> [[b]] -> Maybe [[c]]
```

## Задача 2: Сумма по строкам и столбцам

Напишите функции:

$$s_i = \sum_j A_{ij}, \quad t_j = \sum_i A_{ij}$$

Для суммы по столбцам используйте транспонирование или `foldr (zipWith (+))`.

```
1 rowSums :: Num a    [[a]] -> [a]
2 colSums :: Num a    [[a]] -> [a]
```

## Вариант 13

### 题目 1: 安全的按元素二元映射.

给定二元函数  $f : a \rightarrow b \rightarrow c$ , 对两矩阵做逐元素映射  $C_{ij} = f(A_{ij}, B_{ij})$ . 若行数不同, 或任意一对对应行长度不同, 则返回 `Nothing`. 要求使用 `zipWith` 且不写显式递归.

### 解答

基本思路: 先检查行数一致; 再用 `zipWith` 按行处理, 每行若长度一致则 `Just (zipWith f ...)`, 否则 `Nothing`; 最后用 `sequence` 汇总为一个 `Maybe`.

1. Функция должна вернуть `Nothing`, если число строк различно или хотя бы одна пара строк имеет разную длину.

若行数不同或任意一对对应行长度不同, 函数必须返回 `Nothing`.

2. Проверка `length a /= length b` отсекает несовпадение числа строк на первом шаге.

先用 `length a /= length b` 排除行数不一致的情况.

3. Далее `zipWith go a b` обрабатывает пары строк без явной рекурсии.

再用 `zipWith go a b` 在无显式递归下逐行处理.

4. Внутри `go` мы проверяем длины строк; при совпадении строим `Just (zipWith f ra rb)`, иначе `Nothing`.

在 `go` 中检查行长度; 一致则返回 `Just (zipWith f ra rb)`, 否则返回 `Nothing`.

5. `sequence` превращает список `[Maybe row]` в `Maybe [row]` и автоматически “проваливается” в `Nothing`, если ошибка была хотя бы в одной строке.

`sequence` 把 `[Maybe row]` 聚合为 `Maybe [row]`, 只要有一行失败就整体得到 `Nothing`.

```
1 zipWithMatSafe :: (a -> b -> c) -> [[a]] -> [[b]] -> Maybe [[c]]
2 zipWithMatSafe f a b
3   | length a /= length b = Nothing
4   | otherwise             = sequence (zipWith go a b)
5 where
6   go ra rb
7     | length ra /= length rb = Nothing
8     | otherwise               = Just (zipWith f ra rb)
```

## 题目 2: 行和与列和.

实现行求和  $s_i = \sum_j A_{ij}$  与列求和  $t_j = \sum_i A_{ij}$ . 列和可用转置或 `foldr (zipWith (+))`.

### 解答

基本思路: 行和直接 `map sum`; 列和可先转置再 `map sum`, 因为转置后每一行就是原矩阵的一列.

1. Сумма по строкам:  $s_i = \sum_j A_{ij}$  – это просто сумма элементов каждой строки.

行和  $s_i = \sum_j A_{ij}$  就是每行元素之和.

2. Функция `rowSums = map sum` применяет `sum` к каждой строке.

`rowSums = map sum` 对每行应用 `sum`.

3. Сумма по столбцам:  $t_j = \sum_i A_{ij}$ . После транспонирования столбцы становятся строками.

列和  $t_j = \sum_i A_{ij}$ , 转置后列变为行.

4. Поэтому `colSums = map sum . transpose`.

因此 `colSums = map sum . transpose`.

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3     takeWhile (not . null)
4         . foldr (zipWith (:)) (repeat [])
5
6 rowSums :: Num a => [[a]] -> [a]
7 rowSums = map sum
8
9 colSums :: Num a => [[a]] -> [a]
10 colSums = map sum . transpose
```



Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Фильтрация строк матрицы

Оставьте только те строки матрицы, у которых сумма элементов больше заданного порога  $p$ . Используйте `filter` и `sum`.

```
1 filterRowsBySum :: (Num a, Ord a) => a -> [[a]] -> [[a]]
```

## Задача 2: Норма Фробениуса

Норма Фробениуса:

$$\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$$

Реализуйте через `concat` + `map` + `fold`.

```
1 frobeniusNorm :: Floating a => [[a]] -> a
```

## Вариант 14

### 题目 1: 按行和过滤矩阵.

仅保留行元素和大于阈值  $p$  的行. 要求使用 `filter` 与 `sum`.

### 解答

基本思路: 对每行求和并与阈值比较, 用 `filter` 保留满足  $\sum row > p$  的行.

1. Нужно оставить строки, для которых  $\sum_j A_{ij} > p$ .

需要保留满足  $\sum_j A_{ij} > p$  的行.

2. `filter (\row -> sum row > p)` реализует это напрямую.

`filter (\row -> sum row > p)` 直接按条件过滤.

3. Внутри предиката `sum row` вычисляет сумму элементов строки.

在谓词内部用 `sum row` 求行和.

```
1 filterRowsBySum :: (Num a, Ord a) => a -> [[a]] -> [[a]]
2 filterRowsBySum p = filter (\row -> sum row > p)
```

## 题目 2: Frobenius 范数.

实现  $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$ , 要求用 `concat`、`map`、`fold`.

### 解答

基本思路: 先用 `concat` 展平矩阵, 再对每个元素平方并用 `foldl (+) 0` 求和, 最后开方得到 Frobenius 范数.

1. Норма Фробениуса:  $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$ .

Frobenius 范数定义为  $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$ .

2. `concat a` превращает `[[a]]` в плоский список всех элементов.

`concat a` 把 `[[a]]` 展平成包含所有元素的一维列表.

3. `map (\x -> x*x)` даёт квадраты элементов, а `foldl (+) 0` суммирует их.

用 `map (\x -> x*x)` 得到平方, 再用 `foldl (+) 0` 求平方和.

4. `sqrt` берёт корень из суммы и завершает вычисление нормы.

最后对平方和 `sqrt` 开方得到范数.

```
1 frobeniusNorm :: Floating a => [[a]] -> a
2 frobeniusNorm a =
3   sqrt (foldl (+) 0 (map (\x -> x * x) (concat a)))
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

**Задача 1: MapAccumL / Scanl для частичных сумм**

Реализуйте список префиксных сумм:

$$p_k = \sum_{i=1}^k x_i$$

Используйте scanl.

```
1 prefixSums :: Num a    [a] → [a]
```

**Задача 2: Матрица префиксных сумм (2D prefix)**Для матрицы  $A$  постройте матрицу  $P$ :

$$P_{ij} = \sum_{u \leq i, v \leq j} A_{uv}$$

Подсказка: сначала префиксы по строкам, затем по столбцам (или наоборот).

```
1 prefix2D :: Num a    [[a]] → [[a]]
```

## Вариант 15

### 题目 1: 前缀和.

实现前缀和列表  $p_k = \sum_{i=1}^k x_i$ . 要求使用 `scanl`.

### 解答

基本思路: 用 `scanl (+) 0 xs` 累加得到  $[0, p_1, p_2, \dots]$ , 再用 `tail` 去掉开头的 0.

1. `scanl (+) 0 xs` строит список частичных сумм, начиная с нуля:  $[0, x_1, x_1 + x_2, \dots]$ .

`scanl (+) 0 xs` 会从 0 开始累加, 得到  $[0, x_1, x_1 + x_2, \dots]$ .

2. Так как по определению нужны суммы  $p_k = \sum_{i=1}^k x_i$  без начального нуля, берём `tail`.

按定义需要  $p_k = \sum_{i=1}^k x_i$  而不需要前导 0, 因此取 `tail`.

3. Сложность  $O(n)$ .

时间复杂度为  $O(n)$ .

```
1 prefixSums :: Num a => [a] -> [a]
2 prefixSums xs = tail (scanl (+) 0 xs)
```

## 题目 2：二维前缀和矩阵.

构造  $P_{ij} = \sum_{u \leq i, v \leq j} A_{uv}$ . 提示为先对行做前缀，再对列做前缀.

### 解答

基本思路：二维前缀和可分两步：先对每行做前缀和，再对每列做前缀和；列前缀可通过“转置—行前缀—转置”实现.

1. Определение:  $P_{ij} = \sum_{u \leq i, v \leq j} A_{uv}$ .

二维前缀和定义为  $P_{ij} = \sum_{u \leq i, v \leq j} A_{uv}$ .

2. Если сначала посчитать префиксы по строкам, получим суммы по прямоугольникам в пределах каждой строки.

先做行前缀和可以得到每行内部从左到右的累计和.

3. Затем префиксы по столбцам на уже обработанной матрице добавят накопление сверху вниз, что и даёт сумму по прямоугольнику.

再对列做前缀和就会叠加从上到下的累计，从而得到矩形区域的总和.

4. Префиксы по столбцам удобно реализовать как `transpose . rowPrefix . transpose`.

列前缀可用 `transpose . rowPrefix . transpose` 来实现.

5. Композиция `prefix2D = colPrefix . rowPrefix` соответствует описанным двум шагам.

因此 `prefix2D = colPrefix . rowPrefix` 正好对应“两步法”.

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3   takeWhile (not . null)
4     . foldr (zipWith (:)) (repeat [])
5
6 rowPrefix :: Num a => [[a]] -> [[a]]
7 rowPrefix = map (tail . scanl (+) 0)
8
9 colPrefix :: Num a => [[a]] -> [[a]]
10 colPrefix = transpose . rowPrefix . transpose
11
12 prefix2D :: Num a => [[a]] -> [[a]]
13 prefix2D = colPrefix . rowPrefix
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Map + Maybe (безопасный доступ)

Реализуйте безопасный доступ к элементу списка по индексу. Верните `Nothing` при выходе за границы.

```
1 atMay :: Int -> [a] -> Maybe a
```

## Задача 2: Безопасное сложение матриц

Реализуйте `safeAddMatrices`, возвращающую `Nothing`, если размерности не совпадают.

```
1 safeAddMatrices :: Num a => [[a]] -> [[a]] -> Maybe [[a]]
```

## Вариант 16

### 题目 1: 列表安全索引.

实现安全索引 `atMay`: 若下标越界则返回 `Nothing`, 否则返回 `Just` 对应元素.

### 解答

基本思路: 若  $i < 0$  直接越界; 否则对列表做 `drop i`, 若剩余非空则取首元素, 否则返回 `Nothing`.

1. Безопасный доступ означает: при выходе за границы возвращаем `Nothing`, иначе `Just` элемента.

安全索引要求: 越界返回 `Nothing`, 不越界返回 `Just` 元素.

2. Если  $i < 0$ , индекс точно некорректен, поэтому сразу `Nothing`.

若  $i < 0$ , 下标必然非法, 因此直接返回 `Nothing`.

3. Иначе вычисляем `drop i xs`; если результат имеет вид `(y:_)`, то найден элемент `y`.

否则计算 `drop i xs`; 若结果为 `(y:_)`, 则找到目标元素 `y`.

4. Если после `drop` список пуст, то индекс слишком велик и возвращаем `Nothing`.

若 `drop` 后为空列表, 说明下标过大, 返回 `Nothing`.

```
1 atMay :: Int -> [a] -> Maybe a
2 atMay i xs
3   | i < 0      = Nothing
4   | otherwise =
5       case drop i xs of
6         (y:_) -> Just y
7         []     -> Nothing
```



## 题目 2：安全矩阵加法.

实现 `safeAddMatrices`：若两矩阵维度不一致则返回 `Nothing`.

### 解答

基本思路：把“安全矩阵加法”归约为“安全按元素二元映射”：维度一致则返回 `Just` 的结果矩阵，否则返回 `Nothing`.

1. Сложение матриц – частный случай покоординатного применения бинарной функции  $f$  с  $f = (+)$ .  
矩阵加法是按元素二元映射的特例，只需取  $f = (+)$ .
2. Функция `zipWithMatSafe` уже проверяет размерности и возвращает `Nothing` при несовпадении.  
`zipWithMatSafe` 已经完成维度检查，若不一致就返回 `Nothing`.
3. Поэтому `safeAddMatrices = zipWithMatSafe (+)` напрямую наследует корректность и обработку ошибок.  
因此 `safeAddMatrices = zipWithMatSafe (+)` 直接复用其正确性与错误处理.

```
1 zipWithMatSafe :: (a -> b -> c) -> [[a]] -> [[b]] -> Maybe [[c]]
2 zipWithMatSafe f a b
3   | length a /= length b = Nothing
4   | otherwise             = sequence (zipWith go a b)
5   where
6     go ra rb
7       | length ra /= length rb = Nothing
8       | otherwise               = Just (zipWith f ra rb)
9
10 safeAddMatrices :: Num a => [[a]] -> [[a]] -> Maybe [[a]]
11 safeAddMatrices = zipWithMatSafe (+)
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

**Задача 1: Генерация параметризованной матрицы с условиями**

Сгенерируйте числовую матрицу  $A$  размера  $n \times m$ , элементы которой определяются по формуле

$$A_{ij} = \begin{cases} i^2 + j^2, & \text{если } i \leq j, \\ -(i + j), & \text{если } i > j \text{ и } i + j \text{ чётно,} \\ 0, & \text{если } i > j \text{ и } i + j \text{ нечётно,} \end{cases} \quad i = 0, \dots, n-1, \quad j = 0, \dots, m-1.$$

Требования:

- использовать `list comprehensions` как основной механизм генерации;
- явно использовать индексы строк и столбцов;
- не использовать `map`, `zipWith` и явную рекурсию;
- решение должно быть выражено одной вложенной `list comprehension`.

```
1 genMatrixCond :: Int -> Int -> [[Int]]
```

**Задача 2: Матричное умножение через `zipWith` + `transpose`**

Реализуйте матричное умножение  $AB$  через скалярные произведения:

$$C_{ij} = \langle \text{row}_i(A), \text{col}_j(B) \rangle$$

Используйте `transpose`, `zipWith`, `sum`.

```
1 multMatrices2 :: Num a => [[a]] -> [[a]] -> [[a]]
```

## Вариант 17

### 题目 1: 按条件生成矩阵.

生成  $n \times m$  的整型矩阵  $A$ , 其元素由分段定义给出. 要求仅用一层嵌套列表推导完成, 且不得使用 `map`、`zipWith` 与显式递归.

### 解答

基本思路: 用一层嵌套列表推导同时枚举  $i$  与  $j$ , 并把分段条件翻译成 `if-then-else` 链, 严格按题目给的三种情况赋值.

1. Элемент задаётся кусочно по условиям на  $i$  и  $j$ ; значит, в коде достаточно вложенного `if` с теми же условиями.

元素由  $i$  与  $j$  的条件分段定义, 因此用嵌套 `if` 逐条对应即可.

2. Внешний comprehension перебирает  $i = 0 \dots n - 1$ , внутренний —  $j = 0 \dots m - 1$ .

外层推导遍历  $i = 0 \dots n - 1$ , 内层遍历  $j = 0 \dots m - 1$ .

3. Если  $i \leq j$ , берём  $i^2 + j^2$ .

若  $i \leq j$ , 取  $i^2 + j^2$ .

4. Иначе (то есть  $i > j$ ) проверяем чётность  $i + j$ : при чётном ставим  $-(i + j)$ , при нечётном — 0.

否则即  $i > j$ , 再判断  $i + j$  的奇偶: 偶数取  $-(i + j)$ , 奇数取 0.

5. Такое построение не требует `map`, `zipWith` и явной рекурсии и соответствует требованию условия.

该生成方式不使用 `map`、`zipWith` 与显式递归, 满足题目约束.

```
1 genMatrixCond :: Int -> Int -> [[Int]]
2 genMatrixCond n m =
3   [ [ if i <= j
4       then i ^ 2 + j ^ 2
5       else if even (i + j)
6           then -(i + j)
7           else 0
8     | j <- [0 .. m - 1]
9   ]
10  | i <- [0 .. n - 1]
11  ]
```

## 题目 2: 用 transpose 与 zipWith 实现矩阵乘法.

实现  $C_{ij} = \langle \text{row}_i(A), \text{col}_j(B) \rangle$ . 要求使用 transpose、zipWith、sum.

### 解答

基本思路: 把矩阵乘法写成“行与列的点积”: 先 transpose 得到  $B$  的列, 再用列表推导遍历每行与每列, 点积用 `sum (zipWith (*) ...)`.

1. По формуле  $C_{ij} = \langle \text{row}_i(A), \text{col}_j(B) \rangle$  достаточно уметь брать строки  $A$  и столбцы  $B$ .

由  $C_{ij} = \langle \text{row}_i(A), \text{col}_j(B) \rangle$  可知只需获取  $A$  的行与  $B$  的列.

2. После transpose  $b$  каждая строка результата – это столбец исходной матрицы  $B$ .

对  $b$  转置后, 结果的每一行就是原  $B$  的一列.

3. Далее двойной comprehension перебирает `row <- a` и `col <- bt`.

随后用两层列表推导遍历 `row <- a` 与 `col <- bt`.

4. Скалярное произведение реализуется как `sum (zipWith (*) row col)`.

点积用 `sum (zipWith (*) row col)` 实现.

5. Так получаем матричное умножение без явной индексации по матрицам.

因此无需显式索引即可完成矩阵乘法.

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3   takeWhile (not . null)
4     . foldr (zipWith (:)) (repeat [])
5
6 multMatrices2 :: Num a => [[a]] -> [[a]] -> [[a]]
7 multMatrices2 a b =
8   let bt = transpose b
9   in [ [ sum (zipWith (*) row col) | col <- bt ] | row <- a ]
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Проверка прямоугольности матрицы

Матрица  $[[a]]$  корректна, если все строки одной длины. Реализуйте проверку, используя `all`.

```
1 isRectangular :: [[a]] → Bool
```

## Задача 2: Срез подматрицы

Верните подматрицу, вырезанную по диапазонам строк и столбцов: строки  $[r_0, r_1)$  и столбцы  $[c_0, c_1)$ . Используйте `take/drop`.

```
1 subMatrix :: Int → Int → Int → Int → [[a]] → [[a]]
```

## Вариант 18

### 题目 1: 检查矩阵是否为矩形.

矩阵 `[[a]]` 若每行长度相同则称为“矩形矩阵”. 请用 `all` 实现检查.

### 解答

基本思路: 空矩阵视为矩形; 否则以第一行长度为基准, 用 `all` 检查其余各行长度是否相同.

1. Матрица корректна как прямоугольная, если все строки имеют одинаковую длину.

矩形矩阵要求所有行长度相同.

2. Если матрица пуста, условие выполняется тривиально, возвращаем `True`.

空矩阵可视为平凡满足条件, 因此返回 `True`.

3. Иначе берём длину первой строки и проверяем `all (\row -> length row == length r) rs`.

否则取第一行长度为基准, 用 `all (\row -> length row == length r) rs` 检查其余行.

```
1 isRectangular :: [[a]] -> Bool
2 isRectangular []      = True
3 isRectangular (r:rs) = all (\row -> length row == length r) rs
```

## 题目 2：切片取子矩阵。

返回行范围  $[r_0, r_1)$  与列范围  $[c_0, c_1)$  的子矩阵. 要求使用 `take` 与 `drop`.

### 解答

基本思路：先对行做区间切片 `take (r1-r0) (drop r0 m)`，再对每行做列切片 `take (c1-c0) (drop c0 row)`，整体用 `map` 实现。

1. Диапазон  $[r_0, r_1)$  по строкам означает: отбросить первые  $r_0$  строк и взять следующие  $r_1 - r_0$ .

行区间  $[r_0, r_1)$  等价于先丢弃前  $r_0$  行，再取接下来的  $r_1 - r_0$  行。

2. Это делается как `take (r1-r0) (drop r0 m)`.

对应实现为 `take (r1-r0) (drop r0 m)`。

3. Аналогично для столбцов: в каждой строке делаем `take (c1-c0) (drop c0 row)`.

列切片同理：对每行执行 `take (c1-c0) (drop c0 row)`。

4. `map` применяет операцию по столбцам ко всем выбранным строкам и формирует подматрицу.

`map` 把列切片操作作用到所有选中的行上，从而得到子矩阵。

```
1 subMatrix :: Int -> Int -> Int -> Int -> [[a]] -> [[a]]
2 subMatrix r0 r1 c0 c1 m =
3   map (take (c1 - c0) . drop c0)
4     (take (r1 - r0) (drop r0 m))
```





## Вариант 19

### 题目 1: 行列局部和矩阵.

给定矩阵  $A$ , 对每个位置  $(i, j)$  计算

$$B_{ij} = \sum_{u=1}^i A_{uj} + \sum_{v=1}^j A_{iv} - A_{ij}$$

要求使用 `map`、`scanl`、`zipWith`, 且不得对矩阵做显式索引.

### 解答

基本思路: 把公式拆成行前缀和与列前缀和:  $R_{ij} = \sum_{v=1}^j A_{iv}$ ,  $C_{ij} = \sum_{u=1}^i A_{uj}$ , 则  $B_{ij} = C_{ij} + R_{ij} - A_{ij}$ , 最后用 `zipWith3` 按元素组合.

1. В формуле  $B_{ij} = \sum_{u=1}^i A_{uj} + \sum_{v=1}^j A_{iv} - A_{ij}$  первая сумма – это префикс по столбцу, а вторая – префикс по строке.

公式  $B_{ij} = \sum_{u=1}^i A_{uj} + \sum_{v=1}^j A_{iv} - A_{ij}$  中, 第一项是列前缀和, 第二项是行前缀和.

2. Префиксы по строкам считаем как `tail . scanl (+) 0` для каждой строки, получая матрицу  $R$ .  
行前缀可对每行做 `tail . scanl (+) 0` 得到矩阵  $R$ .

3. Префиксы по столбцам считаем через транспонирование: `colPrefix = transpose . rowPrefix . transpose`, получая матрицу  $C$ .

列前缀可通过转置实现: `colPrefix = transpose . rowPrefix . transpose`, 得到矩阵  $C$ .

4. Далее поэлементно объединяем три матрицы:  $B_{ij} = C_{ij} + R_{ij} - A_{ij}$ ; это делается `zipWith3` по строкам и по элементам.

随后逐元素合成:  $B_{ij} = C_{ij} + R_{ij} - A_{ij}$ , 用 `zipWith3` 在行层与元素层组合.

5. Так мы избегаем явной индексации и выполняем требования задачи.

因此无需显式索引, 满足题目对高阶函数的要求.

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3   takeWhile (not . null)
4     . foldr (zipWith (:)) (repeat [])
5
6 rowPrefix :: Num a => [[a]] -> [[a]]
7 rowPrefix = map (tail . scanl (+) 0)
8
9 colPrefix :: Num a => [[a]] -> [[a]]
10 colPrefix = transpose . rowPrefix . transpose
11
12 rowColSums :: Num a => [[a]] -> [[a]]
13 rowColSums a =
14   let r = rowPrefix a
15       c = colPrefix a
16       combine cr rr ar = zipWith3 (\cv rv av -> cv + rv - av) cr rr ar
17   in zipWith3 combine c r a
```

## 题目 2：按行与按列压缩矩阵.

给定折叠函数  $g$ ，先把每行折叠为一个数  $r_i = g(\text{row}_i(A))$ ，再把每列折叠为一个数  $c_j = g(\text{col}_j(A))$ 。要求使用 `map` 与 `fold`，不得显式递归。

### 解答

基本思路：先用 `foldl1 g` 把每行折叠成  $r_i$ ，再对转置矩阵同样折叠得到  $c_j$ ；空矩阵时列结果定义为空列表。

1. Сжатие по строкам:  $r_i = g(\text{row}_i(A))$  реализуется как `map (foldl1 g) a`.

按行压缩  $r_i = g(\text{row}_i(A))$  可用 `map (foldl1 g) a`.

2. Сжатие по столбцам аналогично, но столбцы удобнее получить транспонированием: `map (foldl1 g) (transpose a)`.

按列压缩同理，但列可通过转置获得：`map (foldl1 g) (transpose a)`.

3. Если матрица пуста, столбцов нет, поэтому корректно вернуть пустой список для  $c$ .

若矩阵为空，则不存在列，因此  $c$  返回空列表是合理的。

4. Таким образом возвращаем пару  $(r, c)$  без явной рекурсии, используя только `map`, `fold` и `transpose`.

最终返回  $(r, c)$ ，全过程仅使用 `map`、`fold` 与 `transpose`，无显式递归。

```
1 transpose :: [[a]] -> [[a]]
2 transpose =
3   takeWhile (not . null)
4     . foldr (zipWith (:)) (repeat [])
5
6 compressMatrix :: (a -> a -> a) -> [[a]] -> ([a], [a])
7 compressMatrix g a =
8   let rowRes = map (foldl1 g) a
9       colRes = case a of
10                [] -> []
11                _ -> map (foldl1 g) (transpose a)
12   in (rowRes, colRes)
```

Фамилия Имя: \_\_\_\_\_

Группа: \_\_\_\_\_

## Задача 1: Fold для композиции функций

Дан список функций  $[a \rightarrow a]$  и значение  $x$ . Постройте результат последовательного применения всех функций:

$$f_n(\dots f_2(f_1(x)) \dots)$$

Используйте `foldr`.

```
1 applyAll :: [a -> a] -> a -> a
```

## Задача 2: Контекстное обновление элементов списка

Пусть дан список значений

$$x = [x_1, x_2, \dots, x_n]$$

и два вспомогательных списка той же длины:

- список коэффициентов  $a = [a_1, \dots, a_n]$ ,
- список смещений  $b = [b_1, \dots, b_n]$ .

Постройте новый список  $y = [y_1, \dots, y_n]$  по правилу

$$y_i = \frac{1}{a_i} \left( b_i - \sum_{j \neq i} x_j \right).$$

То есть каждый элемент обновляется с учётом всех остальных элементов списка.  
Требования:

- использовать функции высшего порядка (`map`, `zipWith`, `fold`);
- не использовать явную рекурсию;
- не использовать индексацию по списку.

## Вариант 20

### 题目 1: 用 foldr 组合函数并应用.

给定函数列表 `[a -> a]` 与初值  $x$ , 返回  $f_n(\dots f_2(f_1(x))\dots)$ . 要求使用 `foldr`.

### 解答

基本思路: 先用 `foldr (flip (.)) id` 把函数列表折叠成一个复合函数  $F = f_n \circ \dots \circ f_1$ , 再计算  $F(x)$ .

1. Требуется получить  $f_n(\dots f_2(f_1(x))\dots)$ , то есть применить функции в порядке списка.

目标是得到  $f_n(\dots f_2(f_1(x))\dots)$ , 即按列表顺序依次作用.

2. Используем `foldr` для построения композиции: `foldr (flip (.)) id [f1,...,fn]` даёт  $f_n \circ \dots \circ f_1$ .

用 `foldr` 构造复合函数: `foldr (flip (.)) id [f1,...,fn]` 得到  $f_n \circ \dots \circ f_1$ .

3. Нейтральный элемент композиции – `id`, поэтому при пустом списке функций результатом будет исходное значение  $x$ .

复合的幺元是 `id`, 因此当函数列表为空时结果就是原值  $x$ .

4. После построения композиции применяем её к  $x$ :  $(f_n \circ \dots \circ f_1)(x)$ .

构造好复合函数后再作用到  $x$ :  $(f_n \circ \dots \circ f_1)(x)$ .

5. Время работы по числу применений функций равно  $O(n)$ .

函数应用次数为  $O(n)$ .

```
1 applyAll :: [a -> a] -> a -> a
2 applyAll fs x = (foldr (flip (.)) id fs) x
```

## 题目 2：上下文相关的列表更新.

给定  $x = [x_1, \dots, x_n]$ , 以及同长度列表  $a = [a_1, \dots, a_n]$ 、 $b = [b_1, \dots, b_n]$ , 构造

$$y_i = \frac{1}{a_i} \left( b_i - \sum_{j \neq i} x_j \right)$$

要求使用高阶函数 `map`、`zipWith`、`fold`, 不得显式递归与索引.

### 解答

基本思路：先用一次求和得到  $S = \sum_{j=1}^n x_j$ , 则  $\sum_{j \neq i} x_j = S - x_i$ , 再用 `zipWith3` 同时计算每个  $y_i$ .

1. Формула обновления:  $y_i = \frac{1}{a_i} \left( b_i - \sum_{j \neq i} x_j \right).$

更新公式为  $y_i = \frac{1}{a_i} \left( b_i - \sum_{j \neq i} x_j \right).$

2. Если заранее вычислить  $S = \sum_{j=1}^n x_j$ , то  $\sum_{j \neq i} x_j = S - x_i$  выражается без повторных суммирований.

若先计算  $S = \sum_{j=1}^n x_j$ , 则  $\sum_{j \neq i} x_j = S - x_i$ , 可避免对每个  $i$  重复求和.

3. Сумму  $S$  получаем через `foldl (+) 0 xs`.

总和  $S$  可用 `foldl (+) 0 xs` 求得.

4. Далее `zipWith3` комбинирует  $(a_i, b_i, x_i)$  и вычисляет  $(b_i - (S - x_i)) / a_i$ .

随后用 `zipWith3` 合并  $(a_i, b_i, x_i)$ , 计算  $(b_i - (S - x_i)) / a_i$ .

5. Так получаем новый список за  $O(n)$  времени и без индексации.

因此在  $O(n)$  时间内、无索引地得到新列表.

```
1 updateContext :: Fractional a => [a] -> [a] -> [a] -> [a]
2 updateContext xs as bs =
3   zipWith3 (\ai bi xi -> (bi - (s - xi)) / ai) as bs xs
4   where
5     s = foldl (+) 0 xs
```